Paradigmas de Programación II

Ingeniería en Informática
Ingeniería en Electrónica
Ingeniería en Telecomunicaciones

Contenidos Mínimos

Fundamentos de Programación Orientada a Objetos (POO)

- Diseño de Clases y Objetos
- Estructuras de Datos en POO
- Métricas de Calidad en POO
- Desarrollo Básico de Aplicaciones en POO
- Reúso y Solución de Problemas de Calidad
- Análisis de Casos de Uso simples de Ingeniería

Metodología

- Exposición participativa dialogada
- Resolución de ejercicios
- Aprendizaje Colaborativo
- Autogestión

Condiciones para Regularidad

- Asistencia: Asistir al 80% de las clases
- Evaluaciones: Aprobar con 4 (55%) o más las actividades evaluativas obligatorias
- Ser estudiante REGULAR (Reglamento del alumno)

Formas e Instrumentos de Evaluación

- Evaluación de Proceso (Guía de TP)
- Evaluaciones parciales (2)
- Evaluación de Integración (Trabajo integrador grupal)
- Régimen de Recuperación (1 (una instancia de recuperación)

Promoción Asignatura

- Evaluación: Examen final (teórico / práctico)
- Aprobación: 4 (55%) en cada parte de la evaluación (teórico / práctico)

Criterios de Evaluación

- Integración de Contenidos
- Vocabulario técnico
- Producción de informes de calidad
- Cumplimiento de requisitos de presentación
- Trabajo colaborativo (Grupo de trabajo)
- Capacidad de análisis, síntesis y búsqueda de datos e información
- Desarrollo de habilidades para el planteo y solución de problemas

Recursos

- Aula Virtual
- JDK (Java)
- Apache NetBeans
- Visual Studio Code
- Otras herramientas

Recursos

- Aula Virtual
- JDK (Java)
- Apache NetBeans
- Visual Studio Code
- Otras herramientas (IA?)

Laboratorio

- Evaluaciones en computadoras de laboratorio

Java

¿Qué es Java?

Java es un lenguaje de programación.

Características:

- Propósito general
- Orientado a objetos
- Multiplataforma.
- Seguro, potente y tiene una comunidad gigante.
- JVM: El código Java no se ejecuta directamente en el procesador, sino en la Máquina Virtual. Permite que funcione igual en Windows, Mac o Linux.

Java

Instalación de Java

JDK (Java Development Kit).

1. **Descarga:** web oficial de Oracle o usar distribuciones libres como *Adoptium (OpenJDK)*.
2. **Configuración:** Instalar y configurar las "Variables de Entorno" (especialmente `JAVA_HOME` y el `Path`).
3. **Verificación:** Abrir la terminal y escribir:
    - java -version
    - javac -version

Java

Compilación y Ejecución en terminal

Java es un lenguaje compilado. El flujo es: **Código Fuente (.java) → Bytecode (.class) → Ejecución.**

Crear el archivo `HolaMundo.java`

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola desde la terminal!");
    }
}
```

En la terminal:

- **Compilar:** `javac HolaMundo.java` (Genera el archivo .class)
- **Ejecutar:** `java HolaMundo`

Java

Errores Sintácticos y Lógicos

- **Errores Sintácticos:** El código rompe las reglas del lenguaje
  a. falta un ;
  b. llave mal cerrada.
  c. escritura incorrecta de sentencia

  El compilador no permite ejecutar el programa.

- **Errores Lógicos:** El programa corre, pero el resultado es incorrecto.
  a. Son los más difíciles de detectar.

  El programa ejecuta pero no devuelve el resultado deseado

Java

Estructura Secuencial

Es el modo más simple: las instrucciones se ejecutan una tras otra, de arriba hacia abajo.

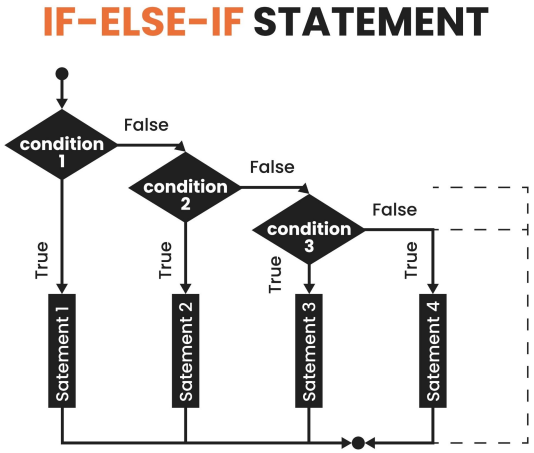
Ejemplo: Calcular el área de un rectángulo.

```
import java.util.Scanner;

public class Factura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Precio del producto: ");
        double precio = sc.nextDouble();

        double iva = precio * 0.21;
        double total = precio + iva;

        System.out.println("IVA: " + iva + " | Total: " + total);
    }
}
```

Java

Estructuras Condicionales (Simples y Compuestas)

Permiten tomar decisiones basadas en una condición booleana.



Simple: Solo ejecuta algo si es verdadero.

```
if (edad >= 18) { System.out.println("Acceso permitido"); }
```

Compuesta: Tiene un camino para el "sí" y otro para el "no".

int nota = 85;

```
if (nota >= 90) {
    System.out.println("Sobresaliente"); // Simple
} else if (nota >= 70) {
    if (nota >= 80) {
        System.out.println("Muy Bueno"); // Anidada
    } else {
        System.out.println("Bueno");
    }
} else {
    System.out.println("Reprobado"); // Compuesta (else)
}
```

Java

Condicionales Anidadas y Operadores Lógicos

Anidadas: Un if dentro de otro if para múltiples niveles de decisión.

Operadores Lógicos:

Permiten evaluar múltiples condiciones en una sola línea.

- && (AND): Ambas deben cumplirse.
- || (OR): Al menos una debe cumplirse.
- ! (NOT): Invierte el valor.

```
int edad = 20;
double promedio = 9.5;
boolean tieneBecaPrevia = false;

if (promedio >= 9.0 && edad < 25 && !tieneBecaPrevia) {
    System.out.println("Candidato elegible para beca completa.");
}
```

Java

Estructuras Repetitivas (While y For)

Permiten repetir un bloque de código.

- **While:** Repite mientras la condición sea verdadera (útil cuando no sabemos cuántas veces repetiremos).

```
int i = 0;
while(i < 3) {
        System.out.println(i);
        i++;
}
```

- **For:** Ideal cuando conocemos el número exacto de iteraciones.

```
for(int i=0; i<5; i++) {
        System.out.println("Contando: " + i);
}
```

Java

Estructura Do While

- A diferencia del while, el do while asegura que el código se ejecute **al menos una vez**
- La condición se evalúa al final.

```
int clave;
do {
    System.out.println("Ingrese su clave:");
    clave = leerDato();
} while (clave != 1234);
```

Java

Cadenas de Caracteres (String)

En Java, **String** no es un tipo primitivo, es una **Clase**.

- texto.length(): Largo de la cadena.
- texto.toUpperCase(): Convertir a mayúsculas.
- texto.equals("Hola"): Comparar contenido (¡no usar == para Strings!).

```
String frase = " Java es Genial ";

System.out.println(frase.trim());          // Quita espacios: "Java es Genial"
System.out.println(frase.length());        // Largo: 16
System.out.println(frase.contains("Java")); // true
System.out.println(frase.substring(1, 5)); // Extrae "Java"
```

Java

Vectores (Arrays)

Un vector es una colección de elementos del mismo tipo bajo un solo nombre.

```
double[] notas = {7.5, 9.0, 6.5, 10.0};
double suma = 0;

for (int i = 0; i < notas.length; i++) {
    suma += notas[i];
}
System.out.println("Promedio: " + (suma / notas.length));
```

Java

Matrices

Estructuras bidimensionales (filas y columnas).

```
int[][] matriz = new int[2][2];
matriz[0][0] = 1; matriz[0][1] = 0;
matriz[1][0] = 0; matriz[1][1] = 1;

// Recorrido con For Anidado
for (int f = 0; f < 2; f++) {
    for (int c = 0; c < 2; c++) {
        System.out.print(matriz[f][c] + " ");
    }
    System.out.println(); // Salto de línea por fila
}
```

Java

Ejercicio 1:

Tipos (Básico - Secuencial y Errores)

Objetivo: Familiarizarse con la lectura de datos, tipos primitivos y el manejo de errores de casting.

- **Reto:** Solicitar al usuario su nombre (String), edad (int) y sueldo deseado (double).
- **Operación:** Calcular cuánto ganaría el usuario en un año aplicando un bono del 15% y mostrar un mensaje formateado: *"Hola [Nombre], con tu edad de [Edad] años, tu sueldo anual con bono sería: [Total]"*.
- **Foco en Java:** Uso de Scanner, System.out.println con concatenación y manejo de decimales.

Java

Ejercicio 2:

Sistema de Acceso con Lógica Booleana (Intermedio - Condicionales)

Objetivo: Aplicar condiciones anidadas y operadores lógicos (&&, ||, !).

- **Reto:** Crear un sistema de acceso a un evento VIP.
- **Reglas:**
  1. Si es mayor de 18 años **y** tiene invitación, accede.
  2. Si es mayor de 65 años, accede aunque no tenga invitación (Socio Honorario).
  3. Si es menor de 18, solo accede si viene con un tutor **y** el tutor tiene invitación.
- **Foco en Java:** Estructura if - else if - else y prioridad de operadores lógicos.

Java

Ejercicio 3:

Cajero (Intermedio - Bucles While / Do-While)

Objetivo: Diferenciar entre bucles que evalúan al inicio y al final.

- **Reto:** Simular el ingreso de un PIN de 4 dígitos.
- **Reglas:** * Usar un do-while para pedir el PIN al menos una vez.
  - El usuario tiene máximo 3 intentos.
  - Si falla los 3, mostrar "Cuenta bloqueada". Si acierta, mostrar "Bienvenido".
- **Foco en Java:** Control de contadores y centinelas dentro de estructuras repetitivas.

Java

Ejercicio 4:

Analizador de Textos (Avanzado - Strings y For)

Objetivo: Manipular la clase String como un objeto y recorrerla.

- **Reto:** Pedir al usuario una frase y una letra al azar.
- **Operación:** 1. Contar cuántas veces aparece esa letra en la frase (sin importar mayúsculas/minúsculas). 2. Invertir la frase completa. 3. Indicar si la frase empieza con la palabra "Java".
- **Foco en Java:** Métodos .charAt(), .length(), .toLowerCase() y .startsWith().

Java

Ejercicio 5:

Radar de Temperaturas (Complejo - Vectores y Matrices)

Objetivo: Manejo de estructuras de datos multidimensionales y algoritmos de búsqueda.

- **Reto:** Crear una matriz de 3x3 que represente las temperaturas de 3 ciudades durante 3 días.
- **Operación:**
  1. Llenar la matriz con datos ingresados por el usuario.
  2. Calcular el promedio de temperatura por cada ciudad (promedio de cada fila).
  3. Encontrar la temperatura más alta de toda la matriz e indicar en qué posición (fila/columna) se encuentra.
- **Foco en Java:** Recorrido de matrices con for anidados y acceso por índices.